

Helix OTA — Master Diagrams Index

Field	Value
Revision	1
Created	2026-06-07
Last modified	2026-06-07
Status	active
Status summary	Index of the six canonical Mermaid (<code>.mmd</code>) diagram sources for the Helix OTA master specs: system architecture, device update sequence, 1.0.0-MVP ER model, 1.0.1 staged rollout, deployment topology, and the build-to-device trust chain. All six were validated with <code>mmdc</code> (<code>mermaid-cli</code>) and render to SVG/PNG. Names and structure are kept consistent with the committed 1.0.0-MVP specs (same table and service names).
Issues	HelixConstitution clause numbers (§11.4.61, §7.1, §11.4.6) are carried from the corpus convention and are UNVERIFIED. <code>staged_rollout.mmd</code> describes the 1.0.1 engine, not the 1.0.0-MVP all-at-once path (so noted in the diagram and below). Reverse-proxy image (<code>caddy:2</code>), dashboard image, and image digests in <code>deployment_topology.mmd</code> are placeholders marked UNVERIFIED per <code>deployment/overview.md</code> §8.
Fixed	N/A (initial revision). Note: during validation two parser-breaking characters were corrected in <code>update_sequence.mmd</code> (a <code>?/;</code> inside a sequence message and a <code>;</code> inside a Note) so all six files now parse cleanly under <code>mmdc</code>.
Continuation	When <code>scripts/export_docs.sh</code> is extended to render standalone <code>.mmd</code> files (it currently extracts and renders only fenced mermaid blocks from Markdown), wire these six into the export run and commit the generated SVG/PNG under <code>_exports/</code>. Add a CI

Field	Value
	step that fails the build if any .mmd here fails mmdc validation.

Table of contents

- 1. [Purpose and scope](#)
- 2. [Diagram catalogue](#)
- 3. [Rendering and export](#)
- 4. [Validation status](#)
- 5. [Consistency with the committed specs](#)
- 6. [Anti-bluff / UNVERIFIED register](#)
- 7. [Sources](#)

The table-of-contents requirement is mandated by HelixConstitution §11.4.61 (UNVERIFIED clause number). This document carries its ToC immediately after the metadata table, per [../documentation_standards.md §3](#).

1. Purpose and scope

This directory holds the **canonical Mermaid diagram sources** for the Helix OTA master specifications. Mermaid is the source of truth for these diagrams per [../documentation_standards.md §6](#); SVG/PNG are derived, never hand-edited.

Each .mmd carries a %% comment header naming its source-of-truth spec section, so the diagram and its prose stay in lock-step. All six are kept consistent with the committed 1.0.0-MVP specs — the **same table names** (12 MVP tables) and the **same service names** (ota-server, postgres, minio, reverse proxy, dashboard).

2. Diagram catalogue

File	Type	What it shows	Primary source spec
system_architecture.mmd	graph TB	Three planes (Dashboard / Control / Device) + data-plane infra + NEW ota-* modules, with the two deliberately extractable seams (OS-adapter, rollout-engine).	2026-06-07-helix-design.md §4
update_sequence.mmd	sequenceDiagram	Device poll → 204 (idle) / 200 (update assigned) branch → download → device re-verify → applyPayload → reboot → boot verify → telemetry, including	../1.0.0-mvp/endpoints.md §12; §5

File	Type	What it shows	Primary source spec
<u>er_model.mmd</u>	erDiagram	the boot-failure auto-rollback path. The 12 MVP tables — users, api_keys, devices, device_groups, device_group_members, artifacts, artifact_versions, releases, deployments, device_deployments, telemetry_events, audit_logs — with attributes and FK relationships (delete behavior annotated).	<u>../1.0.0-mvp/database/schema.r</u> §5
<u>staged_rollout.mmd</u>	flowchart TD	1.0.1 phased rollout (5 % → 10 % → 30 % → 100 %) with health-gated halt-on-failure (halt wins over advance) and operator abort/rollback. Note: 1.0.0-MVP is all-at-once; this is the 1.0.1 engine.	<u>../1.0.1-staged-rollout/README.m</u> master §8
<u>deployment_topology.mmd</u>	flowchart TB	Containerized service set (ota-server, postgres, minio, reverse proxy, dashboard) across edge / app / data networks, on the vasic-digital/containers substrate, with compose vs k8s split.	<u>../1.0.0-mvp/deployment/overv</u> §3–§6
<u>trust_chain.mmd</u>	flowchart LR	Build-pipeline sign → server verify (GATE 1) → device re-verify (GATE 2) → update_engine (GATE 3) → AVB/dm-verity (GATE 4) → A/B boot_control with native rollback. TUF/Uptane MVP-forward seam shown as deferred.	<u>../1.0.0-mvp/security/signing_verificat</u> §2, §4–§7; master §6

3. Rendering and export

These are **standalone .mmd source files**. They render to SVG/PNG with mermaid-cli:

```
mmdc -i system_architecture.mmd -o system_architecture.svg
mmdc -i system_architecture.mmd -o system_architecture.png
```

The corpus export pipeline `scripts/export_docs.sh` converts the Markdown corpus to HTML/DOCX (+ PDF when a LaTeX engine is present) and renders Mermaid diagrams to SVG + PNG. **Accuracy note (§7.1, no bluff):** as of this revision `export_docs.sh` extracts and renders fenced ````mermaid` blocks *embedded in Markdown*; it does not yet iterate standalone `.mmd` files in this directory. Those are rendered directly with `mmdc` (above). Wiring the standalone `.mmd` set into the export run is tracked in this document's Continuation row. The pipeline degrades gracefully: a missing renderer is logged and SKIPPED, never fatal (`../export_pipeline.md`).

4. Validation status

VERIFIED (executed): all six `.mmd` files were parsed and rendered with `mmdc` (`/opt/homebrew/bin/mmdc`, `mermaid-cli`) on 2026-06-07; each produced a non-empty SVG with no parse error. This is positive evidence per §7.1 — the claim “syntactically valid Mermaid” is backed by an actual successful render, not asserted.

To re-validate locally:

```
for f in *.mmd; do mmdc -i "$f" -o "/tmp/${f%.mmd}.svg" && echo
    "OK $f"; done
```

5. Consistency with the committed specs

- **Table names** in `er_model.mmd` are exactly the 12 tables defined in `../1.0.0-mvp/database/schema.md` §4–§5 and created by `migrations/001_initial_schema.up.sql` — no renamed, added, or invented tables. Deferred entities (`deployment_phases`, `rollouts`, `rollback_history`, TUF metadata) are intentionally **not** shown, matching `schema.md` §8.
- **Service names** in `deployment_topology.mmd` (`ota-server`, `postgres`, `minio`, `reverse proxy`, `dashboard`) match `../1.0.0-mvp/deployment/overview.md` §3.
- **Endpoints / status codes** in `update_sequence.mmd` (`GET /api/v1/client/update` → 204/200; `POST /client/telemetry`) match the API spec §12.
- **Gate numbering** in `trust_chain.mmd` (GATE 1–4) matches `../1.0.0-mvp/security/signing_verification.md` §2.

6. Anti-bluff / UNVERIFIED register

Per §7.1 / §11.4.6 (`../documentation_standards.md` §8):

- **VERIFIED (executed):** the six `.mmd` files parse and render under `mmdc` (§4).
- **UNVERIFIED:** HelixConstitution clause numbers (§11.4.61, §7.1, §11.4.6) — carried from corpus convention, not cross-checked against the constitution text.
- **UNVERIFIED:** reverse-proxy image (`caddy:2`), dashboard image, and image digests in `deployment_topology.mmd` (placeholders per `deployment/overview.md` §8).

- **UNVERIFIED:** that `export_docs.sh` renders standalone `.mmd` files — it currently does not (§3); standalone render is via `mmdc` directly.
- No fabricated table names, service names, endpoints, or submodule names are introduced; gated/deferred items are marked, not invented.

7. Sources

- [../2026-06-07-helix-ota-design.md](#) — master design (§4 planes/seams, §5 MVP path, §6 trust model, §8 staged rollout).
- [../documentation_standards.md](#) — metadata/ToC (§2–§3), diagram conventions (§6), anti-bluff (§8).
- [../export_pipeline.md](#) — the real, tested export pipeline (`scripts/export_docs.sh`).
- [../1.0.0-mvp/database/schema.md](#) — 12 MVP tables and relationships.
- [../1.0.0-mvp/deployment/overview.md](#) — service set and topology.
- [../1.0.0-mvp/api/endpoints.md](#) — update-check (204/200) and telemetry endpoints.
- [../1.0.0-mvp/security/signing_verification.md](#) — the four trust gates.
- [../1.0.1-staged-rollout/README.md](#) — staged-rollout engine and halt-on-failure.